



Факультет экономических наук

Базовая кафедра инфраструктуры
финансовых рынков

Москва

Selenium

Автоматизация браузера для сбора данных

Бакланова Валерия Сергеевна

к.э.н., преподаватель БКИФР, младший научный сотрудник ЦФИАНД



Что такое Selenium

2

Selenium — бесплатная библиотека с открытым исходным кодом для автоматизированного тестирования веб-приложений и автоматизации действий в браузере.

Selenium поддерживает все основные браузеры, но в этом курсе мы говорим только про **Chrome** — самый популярный браузер. Работа с другими браузерами отличается только способом установки `webdriver`.

WebDriver — основной продукт проекта **Selenium**: программа, которая напрямую управляет браузером и выполняет в нём действия по командам вашего скрипта.

В курсе мы будем работать с тремя ключевыми сущностями **Selenium**:

- **WebDriver** — управляет браузером, вокруг него строится весь скрипт;
- **WebElement** — абстракция над элементом страницы (кнопка, ссылка, поле ввода);
- **By** — класс со статическими методами для идентификации элементов на странице.



WebDriver — самая важная сущность, отвечающая за управление браузером. Основной ход скрипта строится именно вокруг экземпляра этой сущности.

WebElement — вторая важная сущность: абстракция над веб-элементом (кнопки, ссылки, поля ввода и др.). Это **DOM**-объект, находящийся на веб-странице.

By — класс, содержащий статические методы для идентификации элементов страницы: по **id**, классу, тегу, **CSS**-селектору и другим признакам.



+ Преимущества

- бесплатный продукт с открытым кодом;
- очень гибкий — можно писать большие скрипты, собирающие данные по заданным алгоритмам с множества сайтов;
- разрабатывается с **2004** года — самый популярный инструмент для тестирования веб-приложений;
- огромное сообщество в России и за рубежом;
- много примеров и описаний в открытом доступе.

– Недостатки

- непросто устанавливать — при автообновлении **Chrome webdriver** тоже нужно обновлять;
- подходит для работы только с веб-сайтами;
- может подвисать на слабых машинах — **Chrome** потребляет много оперативной памяти, странице нужно время на загрузку;
- иногда не запускает один и тот же скрипт с первого раза, хотя с повторных запусков всё работает.



```
!pip install selenium
```

```
import selenium
from selenium import webdriver
from selenium.webdriver.common.by import By
```

Минимальный скрипт, который открывает браузер и загружает страницу:

```
with webdriver.Chrome() as browser:
    browser.get(url)
```

with/as — менеджер контекста: сам закрывает браузер по завершении блока, без ручного **driver.quit()**.

ChromeOptions() и headless-режим

6

`ChromeOptions` — класс для настройки параметров и свойств браузера перед его запуском: аргументы командной строки, прокси-сервер, расширения и многое другое.

```
options = webdriver.ChromeOptions()
options.add_argument('--headless')    # без графического интерфейса
options.add_argument("--disable-gpu") # отключает GPU-ускорение

driver = webdriver.Chrome(options=options) # передаем опции в конструктор
driver.get(url)
```

--headless запускает браузер в фоновом режиме: страницы не отображаются на экране, что удобно для автоматизации и веб-скрапинга, особенно на серверах без графической оболочки.



Зачем нужен фоновый режим

7

- **Быстродействие** — нет отрисовки страниц, ниже нагрузка на **CPU** и **RAM**, особенно заметно на слабых машинах.
- **Экономия ресурсов** — окно не занимает место на экране и не мешает текущей работе.
- **Стабильность** — меньше сбоев из-за отсутствия взаимодействия с интерфейсом.
- **Удобство автоматизации** — не нужно открывать и закрывать окна для каждого скрипта.
- **Скрытность** — труднее заметить, что именно делает скрипт — полезно при сборе данных.
- **Меньше капчи** — некоторые сайты менее строго реагируют на запросы из фонового браузера.

Важно: давайте браузеру время дозагрузить содержимое страницы — в фоновом режиме это не всегда заметно.



Полезные флаги **ChromeOptions** (1/2)

8

Флаг	Что делает
<code>--headless</code>	запускает браузер без графического интерфейса
<code>--disable-gpu</code>	отключает аппаратное ускорение GPU
<code>--no-sandbox</code>	запускает браузер без дополнительных мер безопасности
<code>--incognito</code>	режим инкогнито — не сохраняет куки и историю
<code>--window-size=W,H</code>	устанавливает начальный размер окна браузера
<code>--start-maximized</code>	запускает браузер на весь экран
<code>--disable-extensions</code>	отключает все установленные расширения
<code>--user-data-dir=path</code>	директория для хранения профиля пользователя



Полезные флаги **ChromeOptions** (2/2)

9

Флаг	Что делает
<code>--disable-infobars</code>	убирает информационные строки в верхней части окна
<code>--ignore-certificate-errors</code>	игнорирует ошибки SSL -сертификатов
<code>--lang=ru</code>	устанавливает язык интерфейса браузера
<code>--disable-popup-blocking</code>	отключает блокировку всплывающих окон
<code>--disable-notifications</code>	отключает уведомления браузера

```
options.add_argument('--proxy-server=%s' % proxy)  # работа с прокси
```

Полный список флагов **Chrome**: peter.sh/experiments/chromium-command-line-switches



Поиск элементов: локаторы и класс By

10

Локатор — способ идентификации элемента на странице; аргумент, передаваемый методам поиска. Локаторы обеспечивают путь к веб-элементам, необходимым для клика, ввода текста и т. д.

```
from selenium.webdriver.common.by import By
```

By — по сути, перечисление стратегий поиска элементов на странице:

```
ID = "id"                XPATH = "xpath"  
NAME = "name"            TAG_NAME = "tag name"  
CLASS_NAME = "class name"  CSS_SELECTOR = "css selector"
```



Стратегия	Ищет по...	Пример
By.ID	уникальному id	<code>find_element(By.ID, "some_id")</code>
By.NAME	атрибуту name (формы)	<code>find_element(By.NAME, "username")</code>
By.CLASS_NAME	классу	<code>find_elements(By.CLASS_NAME, "btn")</code>
By.TAG_NAME	названию тега	<code>find_elements(By.TAG_NAME, "img")</code>
By.CSS_SELECTOR	CSS -селектору	<code>find_elements(By.CSS_SELECTOR, ".cls")</code>
By.XPATH	выражению XPath	<code>find_element(By.XPATH, "//div[@a='v']")</code>
By.LINK_TEXT	точному тексту ссылки	<code>find_element(By.LINK_TEXT, "Continue")</code>
By.PARTIAL_LINK_TEXT	части текста ссылки	<code>find_element(By.PARTIAL_LINK_TEXT, "Cont")</code>

`.find_element()` и `.find_elements()`

12

`find_element()` возвращает ровно один элемент, `find_elements()` — список найденных элементов.

`.find_element()`

возвращает первый подходящий элемент. Если не найден — исключение **NoSuchElementException**.

`.find_elements()`

возвращает список всех подходящих элементов. Если не найдено — пустой список, без ошибки.

```
# Ищем элемент с тегом img
elements = driver.find_element(By.TAG_NAME, 'img')
elements = driver.find_elements(By.CLASS_NAME, 'some_class')
```



Сценарий 1. Каскадный поиск — сначала находим родительский элемент, затем ищем внутри него дочерний.

```
parent = driver.find_element(By.ID, 'parent_id')
child = parent.find_element(By.CLASS_NAME, 'child_class')
```

Сценарий 2. Поиск внутри списка — проходим по однотипным блокам и ищем элемент в каждом.

```
blocks = driver.find_elements(By.CLASS_NAME, 'block')
for block in blocks:
    block.find_element(By.CLASS_NAME, 'button').click()
```

Сценарий 3. Проверка существования — сначала проверяем, есть ли элементы, и только потом взаимодействуем.

```
elements = driver.find_elements(By.CLASS_NAME, 'some_class')
if elements: elements[0].click()
```

`find_element()` и `find_elements()` возвращают объект(ы) `WebElement` — **DOM**-элемент веб-страницы.

```
url = 'http://parsinger.ru/selenium/3/3.html'
with webdriver.Chrome() as browser:
    browser.get(url)
    elem = browser.find_element(By.CLASS_NAME, 'text')
    print(elem)
```

```
<selenium.webdriver.remote.webelement.WebElement
(session="a86f9c...", element="9F9569...")>
```

WebElement — ваш ключ к манипуляциям с элементом: клику, вводу текста, извлечению атрибутов.



Метод	Что делает
<code>.click()</code>	симулирует клик мышью по элементу
<code>.send_keys("text")</code>	вводит текст в поле (полезно для форм)
<code>.clear()</code>	очищает текстовое поле
<code>.text</code>	возвращает видимый текст элемента
<code>.get_attribute("attr")</code>	возвращает значение указанного атрибута
<code>.is_displayed()</code>	проверяет, отображается ли элемент на странице
<code>.is_enabled()</code>	проверяет, доступен ли элемент для взаимодействия
<code>.is_selected()</code>	проверяет, выбран ли элемент (радиокнопки, чекбоксы)
<code>.submit()</code>	отправляет форму, в которой находится элемент



Заккрытие браузера

16

После завершения работы парсера важно закрыть браузер: при создании **webdriver.Chrome()** в ОС создаётся процесс, который иначе продолжит висеть.

`driver.close()`

закрывает только текущую вкладку/окно браузера.

`driver.quit()`

закрывает все окна, вкладки и процесс **webdriver**.

```
with webdriver.Chrome() as driver:  
    ... # driver.quit() вызовется автоматически
```

Проще всего — использовать менеджер контекста **with/as**: он закроет браузер сам, в нужный момент.



Навигация по истории:

Метод	Что делает
<code>browser.back()</code>	переход на предыдущую страницу
<code>browser.forward()</code>	переход на следующую страницу
<code>browser.refresh()</code>	обновляет текущую страницу

Окно браузера:

Метод	Что делает
<code>browser.maximize_window()</code>	разворачивает окно на весь экран
<code>browser.minimize_window()</code>	сворачивает окно
<code>browser.get_window_size()</code>	возвращает размер окна в виде словаря <code>{'width': ..., 'height': ...}</code>
<code>browser.set_window_size(800, 600)</code>	устанавливает новый размер окна



Метод	Что делает
<code>.get_screenshot_as_file(path)</code>	сохраняет скриншот в файл; True/False по результату
<code>.save_screenshot(name)</code>	сохраняет скриншот в папке проекта
<code>.get_screenshot_as_png()</code>	возвращает скриншот как двоичные данные
<code>.get_screenshot_as_base64()</code>	возвращает скриншот строкой в Base64

Открытие и закрытие:

```
browser.get("http://example_url.ru") # открывает URL
```



Работа с cookies

- `get_cookies()` — список всех **cookies**
- `get_cookie(name)` — **cookie** по имени
- `add_cookie(dict)` — добавляет **cookie**
- `delete_cookie(name)` — удаляет по имени
- `delete_all_cookies()` — удаляет все

Фреймы

- `switch_to.frame(name)` — переключает фокус на фрейм
- `switch_to.default_content()` — возврат к основной странице

JavaScript Alerts

- `switch_to.alert` — переключает фокус на всплывающее окно **JS**

`implicitly_wait(10)` — базовое неявное ожидание элементов (подробнее — дальше в этом разделе).



`.execute_script()`: информация о странице

20

Синтаксис: `webdriver.execute_script(script, *args)`

Скрипт	Что делает
<code>arguments[0].scrollIntoView(true)</code>	прокручивает контейнер, чтобы переданный элемент стал виден
<code>window.open(url, 'tab2')</code>	открывает новую вкладку браузера с именем « tab2 »
<code>document.body.scrollHeight</code>	высота элемента <body> в пикселях
<code>window.innerHeight / innerWidth</code>	высота / ширина окна браузера
<code>alert('текст')</code>	вызывает модальное окно Alert
<code>document.title</code>	заголовок (title) открытого документа
<code>document.readyState</code>	состояние загрузки страницы; « complete », если готова

***args** передаются в **script** и доступны внутри как **arguments[0]**, **arguments[1]** и т. д.



Скрипт	Что делает
<code>document.anchors</code>	список всех ссылок-якорей <code><a></code> на странице
<code>document.cookie</code>	строка со всеми cookies документа, через «;»
<code>document.domain</code>	домен текущего документа
<code>document.forms</code>	список всех форм на странице
<code>window.scrollTo(x, y)</code>	прокручивает документ к указанным координатам
<code>getElementsByClassName('cls')</code>	список элементов с указанным классом
<code>getElementById('id')</code>	элемент с указанным id

```
[x.tag_name for x in browser.execute_script("return document.anchors;")]
```



`window.scrollBy(X, Y)` — прокручивает страницу на **X** пикселей по горизонтали и **Y** по вертикали.

```
with webdriver.Chrome() as browser:
    browser.get('http://parsinger.ru/scroll/1/')
    for i in range(10):
        browser.execute_script("window.scrollBy(0,5000)")
        time.sleep(2)
```

Чтобы долистать до самого низа страницы, не зная её высоты в пикселях, проще прокрутить сразу до конца:

```
window.scrollTo(0, document.body.scrollHeight)
```



Неявное ожидание (Implicit Wait)

23

Implicit Wait — глобальное время, которое **Selenium** ждёт перед тем, как выбросить `NoSuchElementException`, если элемент не найден сразу. **Selenium** периодически (по умолчанию раз в ~0,2 сек) проверяет наличие элемента, пока он не появится или не истечёт тайм-аут.

```
driver.implicitly_wait(2)
driver.find_element(By.ID, 'adder').click() # подождёт до 2 сек
```

Почему это удобно

- **Простота** — достаточно установить один раз в начале скрипта.
- **Глобальность** — применяется ко всем последующим поискам элементов.

Важно помнить

- не рекомендуется смешивать **Implicit** и **Explicit Wait** в одном скрипте;
- на страницах с большим количеством динамики лучше использовать **Explicit Wait**.



Явное ожидание (Explicit Wait)

24

```
from selenium.webdriver.support import expected_conditions as EC
from selenium.webdriver.support.ui import WebDriverWait

WebDriverWait(browser, poll_frequency=0.5, timeout=10) \
    .until(EC.element_to_be_clickable((By.ID, "btn")))
```

poll_frequency — интервал между попытками (0,5 сек по умолчанию); **timeout** — время ожидания в секундах.
.until (method) ждёт, пока **method** не вернёт не-False, иначе — `TimeoutException`.

Условие EC	Когда использовать
<code>element_to_be_clickable</code>	элемент виден и доступен для клика
<code>presence_of_element_located</code>	элемент появился в DOM
<code>visibility_of_element_located</code>	элемент виден на странице
<code>text_to_be_present_in_element</code>	в элементе появился нужный текст
<code>alert_is_present</code>	появилось всплывающее окно Alert
<code>url_contains</code>	URL страницы содержит подстроку



Итоги: шпаргалка по Selenium

25

1

WebDriver

открывает и напрямую управляет браузером — вокруг него строится весь скрипт.

2

By, find_element() / find_elements()

способы поиска элементов на странице по локаторам.

3

WebElement

объект найденного элемента: `.click()`, `.send_keys()`, `.text` и другие методы.

4

ChromeOptions

`headless`-режим и флаги запуска браузера (`--no-sandbox`, `--window-size` и др.).

5

.execute_script()

выполнение произвольного **JavaScript**-кода в браузере.

6

Implicit / Explicit Wait

ожидание появления динамически загружаемых элементов.